

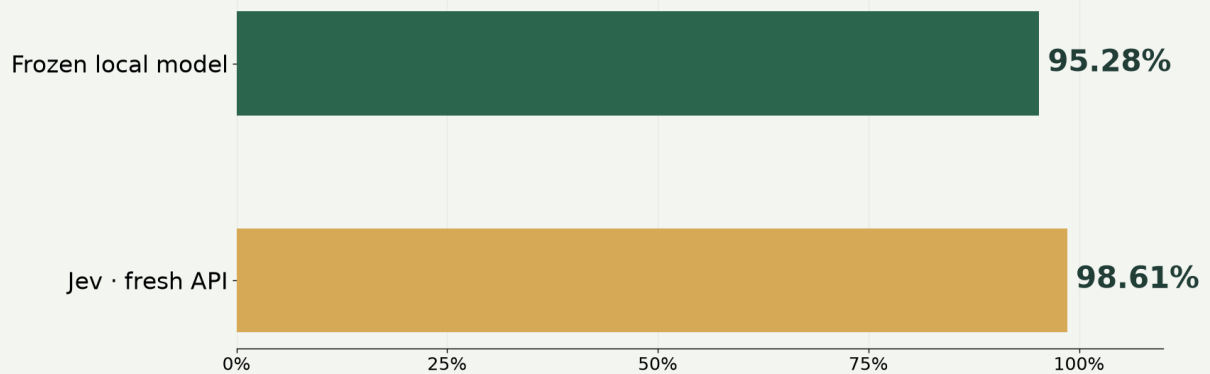
Can Astra Build Its Own Jev?

FILMING GUIDE / 21 SEPTEMBER 2026

The photo-aware local agency, the frozen text challenger, and the measured results.

The stronger model still lost.

360 fresh scenarios · 1,440 policy decisions per engine · zero request failures



Local minus Jev: -3.33 percentage points | 95% interval: -4.38 to -2.29

Synthetic, agent-reviewed references. Two writer-assisted adjudications; no human validation.

Source: round1-deeper-fp16 paired report · frozen before generation · all prespecified cases retained

V2 was not promoted.

The fresh synthetic travel result was 95.28% local versus 98.61% Jev. The app still serves historical V1. Reference repairs and two writer-assisted adjudications are disclosed; no human validation.

1,151 spoken words. 7.4 minutes at 155 WPM, with a planned 9.8-minute runtime including visual holds. Script-only narration source is supplied; no TTS requested.

0:00 - 0:50 / Watch the photo change it

PICTURE

Scene `#demo` at localhost:8770. Start with the live persona demo already visible. Deliver the Jev/local/images claim over the app. Run Maya with the stairs photo by 0:10; remove it and run again. Point at review by 0:25. Briefly show the full agency grid. If either request stalls, preserve real elapsed time and cut the wait transparently.

SAY

I took the idea behind Jev and built a local version for this travel job. And I added image understanding. Watch Maya. The terms fit what she wants, but this photo shows entrance stairs. Her card turns red. Remove the photo, check again, and it asks us to review. It knows less. This runs on my computer. Mine checks holidays. Yours could check a job you repeat every day. I'll show you how the pieces work and how to run my example, even if you've never trained a model.

ON-SCREEN COPY

Watch the photo change it.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

Fresh /api/decide receipts; active V1 plus pretrained vision. Fictional travellers and photos.

0:50 - 1:20 / Give one job a home

PICTURE

Scene `#why` at localhost:8770. Follow holiday input / a small decision / a person checking the reason. Point at the review branch.

SAY

Say you keep asking AI whether a policy fits what a customer wants. You can give that check a home inside your app. It reads the evidence, picks an answer, and shows you why. That's the idea we're borrowing from Jev. You choose the job and the possible answers. Start with ordinary rules and an existing model. Training takes work, so test whether you need it.

ON-SCREEN COPY

Give one job a home.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

Task-selection guidance; no universal price or quality claim.

1:20 - 1:50 / Astra helped build the system

PICTURE

Scene `#build` at localhost:8770. Show the exact prepared specification excerpt, then the saved source and model folders. Identify it as a prepared specification, not a captured chat.

SAY

I asked Astra to help build the travel system, train it and test it. This is the specification. There were follow-ups, fixes and multiple experiments. Once built, the saved models answer these holiday questions without calling Astra. I put the task, app and experiments together. Other people built the foundation models and OpenJev. You'll find their credits in the project.

ON-SCREEN COPY

Astra helped build the system.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

production/ASTRA-BUILD-PROMPT.md; saved source/checkpoint manifests; THIRD-PARTY-NOTICES.md.

1:50 - 2:20 / Three parts. One decision

PICTURE

Scene `#map` at localhost:8770. Point to text reader, pretrained photo reader and app rules in order; keep Maya visible when returning to the agency.

SAY

There are three pieces. A model reads the terms. Another looks at the pictures. Code checks those answers against Maya's wishes and budget. I trained the text reader for travel and connected an already trained image reader. Multimodal means we're using words and pictures. Price comparisons stay in ordinary code. You don't need AI to tell you whether twelve hundred is more than a thousand.

ON-SCREEN COPY

Three parts. One decision.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

travel_lab/vision.py; catalogue.py; active selection.json.

2:20 - 2:50 / Give it three choices

PICTURE

Scene `#labels` at localhost:8770. Click Cash back, then Hotel credit. Point to the mapping between the teaching labels and actual output labels.

SAY

This policy offers hotel credit. Maya wants a cash refund, so it fails her requirement. A clear cash refund passes. If the policy doesn't say, we can't tell. Those are our three labels. A classifier picks a label. That gives our app a short answer it can use. It can still pick the wrong one.

ON-SCREEN COPY

Give it three choices.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

Teaching exercise; actual NLI labels in data/train.jsonl. Fixed format is not a deterministic-output claim.

2:50 - 3:25 / An example has three parts

PICTURE

Scene `#training` at localhost:8770. Open actual train-00000 excerpt. Download the full record. Point at input, question, answer; show the separate held-out file only as data provenance.

SAY

We started with ModernBERT, a model that already understands language patterns. To make it better at this job, we gave it travel examples. Each has three parts. What it reads, what we ask, and the checked answer. This actual record says to contact the booking office. So we can't establish the refund rule. Our examples are made up. For your job, check the labels carefully. Wrong answers teach the wrong lesson.

ON-SCREEN COPY

An example has three parts.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

assets/training-example.json exact data/train.jsonl record; synthetic, no human labels.

3:25 - 3:55 / Practice. Check. Adjust

PICTURE

Scene `#practice` at localhost:8770. Play the two-lane HyperFrame. Hold at the wrong guess, checked label and saved-settings update. Point to unchanged settings in the normal-check lane.

SAY

The model guesses. Training compares that guess with the label and adjusts its saved settings. Repeat with more examples. That's fine-tuning. A prompt gives instructions for a request. Training changes what's saved. Clicking Check in our app uses that saved model; it doesn't train it again. And I've included the travel weights, which are those saved settings, so you can start by running the example.

ON-SCREEN COPY

Practice. Check. Adjust.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

Training code and saved checkpoints. Diagram is conceptual, not a measured individual training step.

3:55 - 4:25 / Keep the exam sealed

PICTURE

Scene `#test` at localhost:8770. Show practice and final-test objects. Open the held-out example and explain its split, then return to the diagram.

SAY

Keep some examples out of training, like an exam the model hasn't practised. Keep related documents together when splitting them. Our later test used made-up cases with agent-reviewed answers, including repairs and disputed answers. There was no human validation. We froze the model before scoring it. Once you inspect those mistakes, you need fresh cases for your next final test.

ON-SCREEN COPY

Keep the exam sealed.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

Frozen V2 protocol; 52 document repairs; two writer-assisted adjudications; hashes. V1 held-out download is labeled separately.

4:25 - 4:55 / Inspect one real mistake

PICTURE

Scene `#failure` at localhost:8770. Show case ...0036 arrival excerpt and both saved outputs. Open full JSON to show the input is longer than the excerpt.

SAY

Here's a real mistake from that test. Arrival at one thirty in the morning is allowed without calling ahead. The reference says yes. Jev says yes. My later model says it can't tell. That sends a clear case to unnecessary review. Both received the same full document. We're looking at an excerpt. This is why I want you to inspect the mistakes too.

ON-SCREEN COPY

Inspect one real mistake.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

assets/benchmark-case.json; raw frozen local and Jev rows for round1-deeper-fp16-0036, arrival.

4:55 - 5:25 / The test kept us honest

PICTURE

Scene `#results` at localhost:8770. Show common-axis bars; point to text-only scope and distinguish demo V1 from later V2. Do not animate a V1 / V2 improvement arrow.

SAY

That later model matched our references about ninety-five percent of the time. Jev was closer to ninety-nine. I didn't beat Jev. It failed our promotion rule, so the demo still uses the earlier model. That one scored sixty-six point five on a different test. These aren't comparable improvement scores. And this chart tests text, not picture understanding. We built the workflow. Its reliability still needs work.

ON-SCREEN COPY

The test kept us honest.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

FROZEN-V2-RESULTS.md; qualification-decision.json; V1 summary. No model or final-test changes.

5:25 - 6:00 / Give the system eyes

PICTURE

Scene `#vision` at localhost:8770. Play photo / observation / app. Open photo-lab.html, choose a local image, run it and inspect actual answers and image hash. Keep the service running before filming.

SAY

Now give the system eyes. This pretrained reader looks at the actual pixels and answers questions about visible features. I connected it through OpenJev. Pick a PNG or JPEG in the photo lab and run it on this computer. We send the picture, without its filename or an expected answer. This model reads pictures. The holiday illustrations were created separately.

ON-SCREEN COPY

Give the system eyes.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

Fresh /api/observe receipt; fixed loopback endpoint; pretrained DiffusionGemma. Generated fictional catalogue photos.

6:00 - 6:30 / A picture leaves things out

PICTURE

Scene `#boundary` at localhost:8770. Show pond, ask the three questions and reveal the conclusion. Refer back to the missing stairs photo.

SAY

There's water here, but that doesn't establish a swimming pool. Even a pool photo can't tell us whether access costs extra. Pictures show visible features. Terms tell us what's promised. Our forty offers share nine fictional images, so this isn't a test of forty real hotels. And removing Maya's stairs photo never proves the entrance is accessible.

ON-SCREEN COPY

A picture leaves things out.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

Generated-image ledger; location-photos.json; image evidence limitations.

6:30 - 7:00 / The app makes the call

PICTURE

Scene `#rules` at localhost:8770. Play budget/terms/photo gates. In the actual agency open a traveller drawer and show the clause/photo that caused decline or review.

SAY

The app combines the checks. Budget passes. Terms pass. Stairs fail, so Maya gets a decline. Missing evidence goes to review unless another requirement already fails. Open her card and inspect the reason. A green card means these checks passed according to our models. A person still confirms the booking details. The app doesn't book anything.

ON-SCREEN COPY

The app makes the call.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

catalogue.verdict; apply_visual_requirements; live reason drawer; companion local API example.

7:00 - 7:30 / Check your computer first

PICTURE

Scene `#setup` at localhost:8770. Show tested M5 Max/128 GB host and separate text/image requirements. Open Start here prerequisites briefly.

SAY

Check your computer before downloading. I used an M5 Max with a hundred and twenty-eight gigabytes of memory. That's the tested machine, not a minimum. The supplied image setup needs Apple silicon and about sixteen gigabytes of model downloads. That's disk space, not a RAM requirement. Smaller machines and Windows or Linux image setups aren't validated here. Text and pictures have separate setup steps.

ON-SCREEN COPY

Check your computer first.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

Inspected host, macOS 26.5.2; pinned vision launcher. No unsupported hardware promises.

7:30 - 8:10 / Run it before changing it

PICTURE

Scene `#first-run` at localhost:8770. Open Start here. Show source and restored files, run `first_run.py`, start agency in a spare terminal if needed, inspect service status and run a fresh holiday. No fake installation animation.

SAY

Open Start here. Get the source and matching model archive, then install the listed tools. Run the first-run check to see what's missing. Start the text service, open the app, and add the photo service when ready. The commands are copyable. Check a holiday yourself. Leave the services running; after sleep, check them again. If they're unavailable, the page tells you instead of showing an old answer.

ON-SCREEN COPY

Run it before changing it.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

`tools/first_run.py`, `tools/restore_companion.py`, `docs/SETUP.md` and live `/api/status`. Public access separately gated.

8:10 - 8:40 / Count the whole bill

PICTURE

Scene `#cost` at localhost:8770. Show build/setup and local-running cost columns, then separately labeled historical timings. Point to fresh demo time rather than implying benchmark-equivalent workloads.

SAY

These local checks don't create a model-provider API bill. You still have hardware, electricity, setup and maintenance. Building and training can cost money too. I haven't measured the whole bill or proved a fixed saving. These timings cover different jobs, so compare the same workload on your computer before deciding whether this approach pays off.

ON-SCREEN COPY

Count the whole bill.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

Historical V1 HTTP timing receipts and photo integration scan; no measured total-cost ratio.

8:40 - 9:15 / Change one example yourself

PICTURE

Scene `#repurpose` at localhost:8770. Edit support request, choose its human label, add it and download JSONL. Run `prepare_task.py` on the saved sample. Show held-out separation in the adaptation guide.

SAY

Say your job is sorting support messages. Here's someone charged twice. Choose Billing, add the example, and download the data file. You've made a checked example. You haven't trained a support model yet. The guide shows how to prepare the data, change the task and keep a separate test. Start with cases you understand, including confusing ones. Changing the labels alone won't teach the travel model a new job.

ON-SCREEN COPY

Change one example yourself.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

Real JSONL export; `tools/prepare_task.py`; `docs/ADAPT-YOUR-OWN.md`. No completed support-model claim.

9:15 - 9:45 / Start with a working example

PICTURE

Scene `#take-it` at localhost:8770. Open Start here and the question index. Show current access/permission status accurately; use only the delivered link once verified. Keep original creator/model credits visible.

SAY

You can start with the travel demo, inspect an answer, and then adapt the approach to a job you understand. The same repository brings together the code, setup guide, examples, model downloads and results, including the losses. Start with the guide and check the reuse terms for each part. Get one result running on your computer, then change one thing at a time.

ON-SCREEN COPY

Start with a working example.

EDITING NOTE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Setup commands and photo results must be real; extended installation waits can be cut with the elapsed time disclosed.

SOURCE / TRUTH CARD

One repository stays private until the video goes live. Verify signed-out source and model-download access before adding the description link. Original-code reuse terms still need a decision. Retain all upstream credits.

Before the take

Start the OpenJev vision service on localhost:8081 and the agency on localhost:8765. Reset with R. Select Four friends. Run Atlantic hideaway fresh. P pauses or resumes the presentation. Escape closes details. Warm both models before recording; cold load is separate from warm request timing.

The charts show saved benchmark results. The agency serves V1, while the final challenger result is V2. Photos, travellers and offers are fictional. Source-clause linkage is structured application logic.

Truth boundaries

Final references are synthetic and agent-reviewed, with 52 document repairs and two non-unanimous writer-assisted adjudications. All 360 scenarios remain. No human validation. The public workflow lane measures teacher agreement; upstream exposure may apply. Thirty-five workflow decisions failed the frozen context limit and remain in the denominator. Official TypeSafe benchmark scores were not reproduced.

Measured agency timing

Active V1: cold first HTTP request 10.1 seconds including loading. Three fresh forty-offer scans took 4.0, 4.2 and 4.5 seconds. Warm HTTP median 105 ms, 121 unique fresh requests, zero failures. These are text-only historical timings, separate from V2 model-only latency. The photo-aware 40-offer integration scan took 40.7 seconds in its recorded run.

Companion resource

Show the verified local bundle, including both checkpoints and original audit evidence. Add a public download link only once separately arranged. The OpenJev image branch ran locally; its pinned model is downloaded separately. The earlier text-only diffusion comparison was not executed. No public upload, deployment or messaging is implied.